

Campus Eventos

Aplicación de Consulta de Eventos Universitarios
Desarrollo de Interfaz en Flutter

Andrew

16 de septiembre de 2026

Índice

1. Introducción	3
2. Desarrollo	3
2.1. Estructura del proyecto	3
2.2. Widgets utilizados	3
2.3. Funcionamiento general	4
3. Implementación	4
3.1. Filtrado de eventos	4
3.2. Manejo de estado	4
3.3. Diseño responsivo	5
3.4. Organización del código	5
3.5. Tema visual Gruvbox	5
4. Resultados	5
5. Conclusiones	7
5.1. Aprendizajes	7
5.2. Problemas encontrados y soluciones	7
6. Anexos	8
6.1. Repositorio del proyecto	8
6.2. Tecnologías utilizadas	8
6.3. Categorías y eventos	8

1. Introducción

El presente documento describe el desarrollo de **Campus Eventos**, una aplicación móvil construida con Flutter cuyo objetivo es permitir a los estudiantes universitarios consultar las diferentes actividades que se realizan dentro de la institución.

La aplicación ofrece una interfaz visualmente atractiva basada en la paleta de colores *Grubbox Dark*, organizada por categorías (Académicos, Deportivos, Culturales, Tecnología y Talleres), con un sistema de filtrado en tiempo real y diseño responsivo que se adapta a distintos tamaños de pantalla.

2. Desarrollo

2.1. Estructura del proyecto

El proyecto sigue una arquitectura limpia y organizada en carpetas temáticas dentro del directorio `lib/`:

```
lib/  
+-- main.dart  
+-- data/  
    +-- event_data.dart  
+-- screens/  
    +-- home_page.dart  
+-- widgets/  
    +-- event_card.dart  
    +-- category_chip.dart  
+-- theme/  
    +-- app_theme.dart
```

Cada carpeta tiene una responsabilidad clara: `data/` contiene los datos de los eventos, `screens/` alberga las pantallas principales, `widgets/` los componentes reutilizables y `theme/` la configuración visual de la aplicación.

2.2. Widgets utilizados

La aplicación utiliza los siguientes widgets de Flutter, cumpliendo con todos los requisitos de la práctica:

- **StatefulWidget**: `HomePage` como widget principal con estado.
- **setState()**: para actualizar la categoría seleccionada y re-renderizar la interfaz.
- **GridView.builder**: para mostrar los eventos en una cuadrícula adaptable.
- **Card**: como contenedor de cada evento en la tarjeta `EventCard`.
- **ListView.builder**: para el scroll horizontal de las categorías (chips de filtro).
- **ChoiceChip**: implementado en `CategoryChip` para la selección de categorías.
- **SnackBar**: para mostrar confirmación al presionar “Me interesa”.
- **Image.network**: para cargar imágenes de los eventos desde la red.

- **Row / Column:** para la organización horizontal y vertical de los elementos.

Otros widgets complementarios incluyen `AppBar`, `SafeArea`, `Padding`, `LayoutBuilder`, `Expanded`, `Flexible` y `FilledButton.tonalIcon`.

2.3. Funcionamiento general

La aplicación inicia en `main.dart`, que ejecuta `CampusEventosApp`, un `StatelessWidget` que configura el `MaterialApp` con el tema personalizado `Gruvbox` y carga la pantalla principal `HomePage`.

La pantalla principal presenta:

1. Un encabezado con el título “Eventos universitarios” y una descripción general.
2. Una lista horizontal de categorías filtrables mediante chips.
3. Un contador de “Eventos encontrados” que se actualiza dinámicamente.
4. Una cuadrícula de tarjetas de eventos que muestra imagen, nombre, categoría, fecha, hora, lugar y cupo de cada actividad.
5. Un botón “Me interesa” en cada tarjeta que activa un `SnackBar` con el nombre del evento.

3. Implementación

3.1. Filtrado de eventos

El filtrado se implementa en el método `build` de `HomePage`. La variable `categoriaSeleccionada` almacena la categoría activa y se evalúa con una expresión condicional:

```
1 final eventosMostrados = categoriaSeleccionada == 'Todos'
2   ? eventos
3   : eventos.where((e) => e['categoria'] == categoriaSeleccionada)
   .toList();
```

Cuando se selecciona “Todos”, se muestran los 18 eventos. Al seleccionar una categoría específica, se filtra la lista usando `where()` y se muestra únicamente los eventos correspondientes. El contador de “Eventos encontrados” se actualiza automáticamente al usar la longitud de `eventosMostrados`.

3.2. Manejo de estado

El estado se gestiona de forma nativa con `StatefulWidget` y `setState`. La variable `categoriaSeleccionada` es el único campo de estado, lo cual es adecuado para la complejidad de esta aplicación. No se utilizaron paquetes externos de gestión de estado (`Provider`, `BLoC`, `Riverpod`), ya que el alcance del proyecto no lo requiere.

El flujo de datos sigue un patrón ascendente-descendente:

- Los datos (`evento`) y callbacks (`onPressed`) se pasan **hacia abajo** a `EventCard`.

- Las selecciones (`onTap`) se propagan **hacia arriba** al widget padre.

3.3. Diseño responsivo

La adaptación a distintos tamaños de pantalla se logra mediante `LayoutBuilder` en el cuerpo de la pantalla principal. Se definen tres breakpoints:

```
1 int crossAxisCount;  
2 if (constraints.maxWidth >= 900) {  
3   crossAxisCount = 4;  
4 } else if (constraints.maxWidth >= 600) {  
5   crossAxisCount = 3;  
6 } else {  
7   crossAxisCount = 2;  
8 }
```

Adicionalmente, se utiliza `mainAxisExtent: 370` en el `SliverGridDelegateWithFixedCrossAxisCount` para garantizar que las tarjetas no se desborden en ningún ancho de pantalla. Los textos largos se manejan con `maxLines` y `TextOverflow.ellipsis`.

3.4. Organización del código

El proyecto se divide en cuatro capas claras:

- **Datos** (`event_data.dart`): Lista constante de 18 eventos y 6 categorías, sin dependencias de widgets.
- **Pantallas** (`home_page.dart`): Lógica de presentación y estado.
- **Widgets** (`event_card.dart`, `category_chip.dart`): Componentes reutilizables y modulares.
- **Tema** (`app_theme.dart`): Paleta de colores Gruvbox completa y configuración de `ThemeData` con Material 3.

3.5. Tema visual Gruvbox

La propuesta visual propia se basa en la paleta de colores *Gruvbox Dark*, una paleta popular en la comunidad de terminales y editores de código. Se implementó una clase `GruvboxPalette` con más de 30 colores estáticos y un `ColorScheme` personalizado de Material 3, aplicándose a `AppBar`, tarjetas, chips, botones y snack bars.

4. Resultados

A continuación se presentan capturas de pantalla de la aplicación funcionando en distintos tamaños de pantalla.

Las capturas se obtuvieron ejecutando la aplicación en un dispositivo móvil y en una ventana ampliada de escritorio.

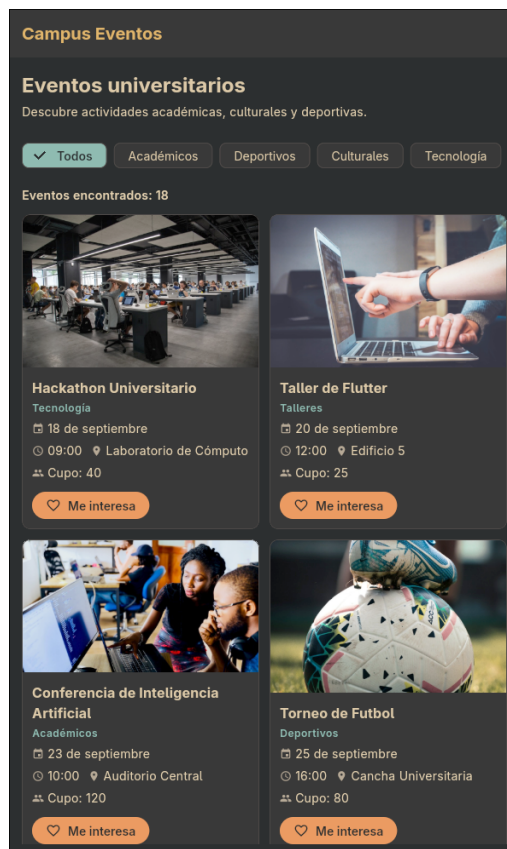


Figura 1: Vista de la aplicación en dispositivo móvil (cuadrícula de 2 columnas).

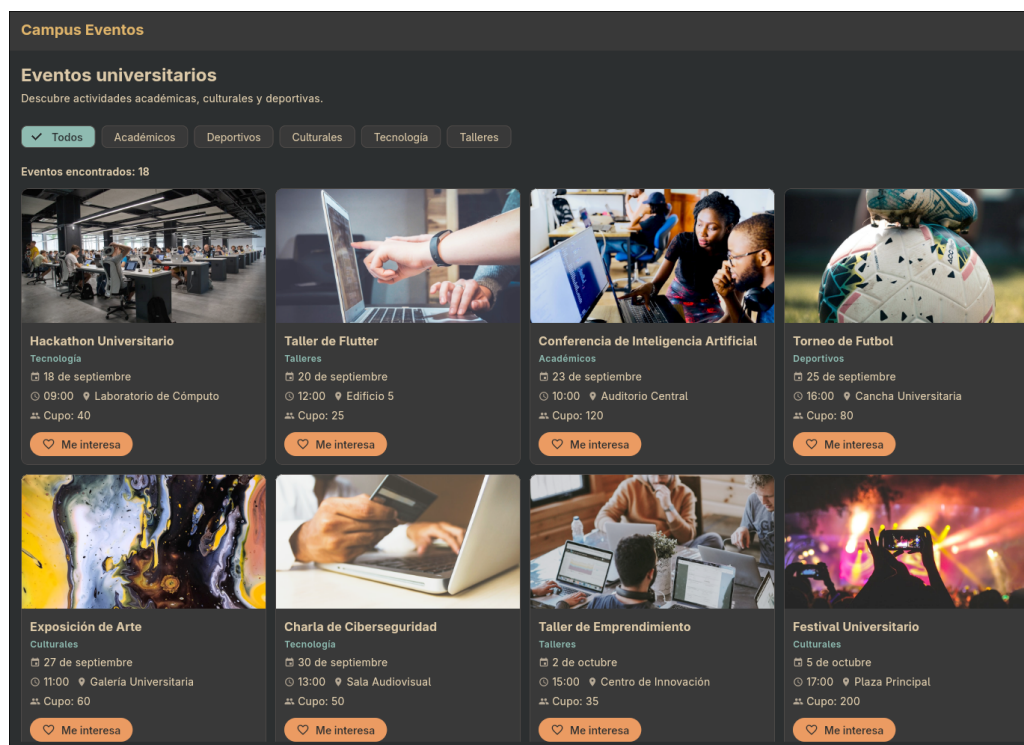


Figura 2: Vista de la aplicación en pantalla ampliada (cuadrícula de 4 columnas).

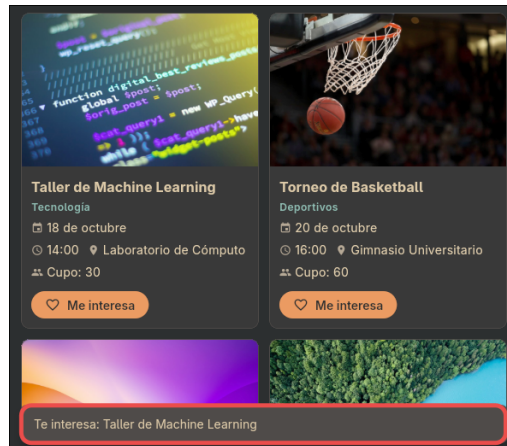


Figura 3: Snackbar de confirmación al interactuar con un evento.

5. Conclusiones

5.1. Aprendizajes

Durante el desarrollo de esta práctica se consolidaron los siguientes conocimientos:

- Uso correcto de `StatefulWidget` y `setState` para la gestión de estado simple.
- Construcción de interfaces modulares mediante widgets reutilizables (`EventCard`, `CategoryChip`).
- Implementación de diseños responsivos con `LayoutBuilder` y breakpoints dinámicos.
- Organización del código en capas para mantener la separación entre datos, presentación y estilos.
- Creación de un tema visual personalizado con Material 3 y una paleta de colores externa.
- Manejo de imágenes de red con `errorBuilder` para evitar errores en pantalla.

5.2. Problemas encontrados y soluciones

1. **Desbordamiento de tarjetas en pantallas pequeñas:** Se solucionó fijando `mainAxisExtent: 370` en el delegado del `GridView`, garantizando que cada celda tenga una altura fija independiente del contenido.
2. **Textos largos que rompían el diseño:** Se aplicó `maxLines` con `TextOverflow.ellipsis` en títulos y descripciones, junto con `Flexible` y `Expanded` para distribuir el espacio restante.
3. **Fallo al cargar imágenes de red:** Se implementó un `errorBuilder` en `Image.network` que muestra un ícono de respaldo (`Icons.image_not_supported`) sobre un fondo de color cuando la imagen no está disponible.
4. **Filtrado que no actualizaba el contador:** Se verificó que el contador “Eventos encontrados” estuviera referenciando la lista ya filtrada (`eventosMostrados.length`) y no la lista completa.

5. **Color del texto en chips de categoría:** Se ajustó el tema de `ChoiceChip` en `AppTheme` para asegurar la legibilidad del texto sobre el fondo seleccionado, resolviendo un problema de contraste con la paleta `Gruvbox`.

6. Anexos

6.1. Repositorio del proyecto

El código fuente completo de la aplicación está disponible en GitHub:

<https://github.com/Andrew869/flutter-app-eventos>

6.2. Tecnologías utilizadas

- **Flutter SDK:** Framework de desarrollo multiplataforma.
- **Dart:** Lenguaje de programación.
- **Material Design 3:** Sistema de diseño visual.
- **Paleta Gruvbox Dark:** Esquema de colores personalizado.

6.3. Categorías y eventos

Categoría	Número de eventos
Todos	18
Académicos	3
Deportivos	3
Culturales	4
Tecnología	4
Talleres	4

Cuadro 1: Distribución de eventos por categoría.